PreAct: A Verified Self-Extending Executable Code Corpus for Computer-Using Agents

PreAct Authors[1]
19pine.ai

June 14, 2026

Abstract

Computer-Using Agents (CUAs) re-derive solutions to familiar tasks on every invocation, paying full LLM-inference cost even for behavior they have demonstrably completed before. We present PreAct, a CUA harness that grows a *self-extending executable code corpus*: state-machine programs that are simultaneously the compiled artifact of a successful trajectory *and* the runtime program for subsequent invocations. **The artifact is the runtime**; what the agent "remembers" is what it can directly execute. The harness is a verified compile-extend loop driven by RPA-replay-failure detection: when CUA succeeds, the live trace is compiled into a state-machine program; when a later replay fails or only partially replays, the harness falls through to CUA, generates a fresh state machine, and—if it passes a *verify-before-store gate*—extends or replaces the corpus.

We validate the load-bearing claim—that the verify-before-store gate is empirically *necessary* for monotonicity—at multi-seed paper-grade rigor on *three* structurally different platforms. A 2×2 ablation (cold/warm × gate ON/OFF) gives a diff-of-deltas of **2.6 tasks** on AndroidWorld official-15 (n=5 seeds, paired sign-test $p \approx 0.031$), **2.6 tasks** on OSWorld test_tiny (n=5 reps, paired sign-test $p \approx 0.031$), and **1.75 tasks** on a 12-task WebArena shopping_admin subset (n=4+4 symmetric). The cross-platform meta-test on 14 paired observations all favoring the gate or tied (13 non-tie, all in favor of gate-ON) yields $p \approx 1.2 \times 10^{-4}$ under the no-effect null. Under gate-OFF the lossy-replay mechanism is fully observable: across Android and OSWorld combined we document five reproducible cov=100%/score=0 warm replays; on WebArena gate-OFF *all 48 warm replays score 0* across 4/4 reps. The gate either intercepts these at compile time (the WebArena harness with the gate ON deletes 4–5 lossy compiles per rep before storage) or, on Android/OSWorld, prevents them from entering the corpus in the first place. Cold→warm monotonicity itself holds across three Gemini seeds (mean $\Delta = +1.33 \pm 0.58$); a cross-model replication with Claude Sonnet 4.6 reaches the same 73.3% (11/15) on official-15 and reproduces the identical stable failure set.

Equally informative are the *negative* findings: code-level runtime guardrails are aggregate-neutral at n=5 (cold means identical, 10.2 = 10.2); prompt-level Pre-Act content is dead code in the production CUA path; and even a tuned embedding-RAG selector (MiniLM or bge-large) equals or exceeds the agentic LLM selector on the 58-program corpus, reaching 100% functional retrieval against the agentic selector's 75.6%. On WebArena's same-task replay protocol, a matched n=4+4 head-to-head against *Muscle-Mem* [5]—a verbatim-action-cache baseline with full-CUA fallback on cache miss—initially exposed a 3.25-task warm-SR gap (Muscle-Mem $\Delta = -0.75 \pm 1.50$ vs. PreAct gate-ON -4.0 ± 2.94). Diagnosing the mechanism (gate-reject leaves Run 2 with no artifact; Muscle-Mem's cache-miss falls through to fresh CUA) and adding a $\sim$ 30-line cache-miss-to-CUA fallback to the harness closes the gap completely: **PreAct gate-ON + fallback achieves warm-$\Delta = -1.0 \pm 1.83$, statistically indistinguishable from Muscle-Mem (Welch's two-sided t-test $p \approx 0.84$)**, while preserving the gate's Android/OSWorld monotonicity unchanged. The harness has two cooperating load-bearing pieces—verify-before-store gate and cache-miss-to-CUA fallback—and neither prompts, guardrails, nor selector implementation choice is load-bearing.

1 Introduction

1.1 The Rerun Crisis

Computer-Using Agents based on the Observation–Reasoning–Action paradigm [9, 1] re-derive their solutions to familiar tasks at every invocation. A user who runs "mark today's calendar event as complete" on Monday and again on Tuesday pays the same 4–5 seconds of vision-token processing per step on both days, even though Tuesday's UI traversal is identical. We call this the *rerun crisis*: $O(M \times N)$ LLM cost on tasks the agent has already solved, where M is the user's daily task count and N is the per-task LLM call count.

Two prior research directions attack this gap. **Skill systems** (e.g., SAGE [4]) save agent behaviors as parameterized skills; the skills still require an LLM-bound loop at execution time, retaining most of the per-step inference cost. **Compilation systems** (Compiled-AI [7], ActionEngine [10]) compile workflows into deterministic code; the deterministic-code regime targets narrow business-logic functions or, in ActionEngine's case, generates flat Python scripts from a state machine that is built via untargeted application crawling rather than goal-directed task execution. Concurrent record/replay systems—Muscle-Mem [5], AgentRR [3], Workflow-Use [2]—store linear action sequences with agent fallback if the cache misses; the stored representation has no formal state verification, no conditional branching, and no incremental refinement.

1.2 PreAct's Position

PreAct introduces a different commitment: **the artifact is the runtime**. State-machine programs in PreAct's corpus are simultaneously the compiled artifact of a successful trajectory *and* the directly-executable runtime program. There is no flat-script generation step (cf. ActionEngine), no LLM-bound execution loop (cf. skill systems), no linear-list fragility (cf. Muscle-Mem). When the harness retrieves a state-machine program for a task, it walks the graph: each state's verification predicate is checked against the live UI, and each transition's action is executed. If a verification fails or an action errors, the harness falls through to CUA, generates a fresh state machine from the live trace, and—if it passes the verify-before-store gate—extends or replaces the corpus.

The corpus thus *self-extends*: the harness's static code defines the compile-extend-replace loop, but the executable state-machine corpus grows monotonically through verified compile cycles. This is closer to a self-modifying executable code corpus than to a passive cache of past actions.

1.3 Contributions

We present and empirically validate the following contributions:

1. **State-machine-as-executable architecture** (§3). The state-machine program produced by compilation *is* the runtime artifact: the harness walks the graph at execution time rather than running generated flat Python.
2. **Self-extending executable code corpus** (§3, §4). The corpus grows through a verified compile-extend-replace loop driven by RPA-replay-failure detection.
3. **Verify-before-store gate** (§3, §4.3). A *double gate* (`replay.success` $\wedge$ `score` ≥ 1.0) filters lossy compiles before they enter the corpus. *Empirically necessary for monotonicity*: an n=5+5+4 AB ablation across three structurally different platforms yields a meta-test

Table 1: PreAct vs. related approaches.

System	Memory representation	Replay fidelity	Fallback path	Self-extension
Muscle-Mem	Linear tool calls	Direct	Agent on cache miss	Append cache only
AgentRR	Multi-level experiences	Indirect	LLM	Append only
Workflow-Use	Linear scripts	Direct	Agent	Append only
ActionEngine	State machine via crawl	Flat-Python generated	None	Re-crawl from scratch
PreAct	**State machine**	**The SM IS the runtime**	**CUA + recompile under verify-gate**	**Verified corpus growth + dedup-replacement**

$p \approx 10^{-4}$, with reproducible cov=100%/score=0 smoking-gun replays observed at warm time on Android and OSWorld and at compile time (gate-rejection events) on WebArena.

4. **Cache-miss-to-CUA fallback** (§4.5.1). When the verify-gate empties the corpus for a particular task, Run 2 runs a fresh CUA exploration instead of returning "no artifact". This $\sim$ 30-line harness fix closes a 3-task warm-SR gap against *Muscle-Mem* on WebArena (warm-Δ $-4.0 \rightarrow -1.0$, matching Muscle-Mem's -0.75) without changing the gate's Android/OSWorld behavior.
5. **Multi-seed, cross-model monotonic refinement on AndroidWorld** (§4.2). Cold$\rightarrow$warm refinement holds across n=3 Gemini 3 Flash seeds with the rag_db reset per seed (mean $\Delta = +1.33 \pm 0.58$); a cross-model replication with Claude Sonnet 4.6 reaches the same 73.3% (11/15) on official-15 with an identical stable failure set. (The cross-*platform* extension of the gate's necessity claim is contribution 3 above.)

The data refute two paper-v1 implications: prompt-level Pre-Act content is dead code in production (§4.7), and code-level runtime guardrails are statistically aggregate-neutral at n=5. **The harness is the contribution; prompts and guardrails are not load-bearing.**

2 Related Work

We compare PreAct against four classes of prior work: pure CUA baselines, skill-based systems, compilation-based systems, and concurrent record-replay systems. Table 1 summarizes the comparison along five dimensions.

The architectural commitment that distinguishes PreAct is the second column (*the SM is the runtime*) combined with the fourth (*recompile under verify-gate*). ActionEngine builds a state machine but emits flat Python from it; the state machine is consumed at compile time and the runtime is the generated script. PreAct executes the state machine directly: each state's verification predicate is evaluated against the live UI, and each transition is dispatched to the underlying action backend (XPath click, JSON action, etc.). This direct execution enables (a) per-state verification and graceful fallback, (b) in-place patching when a transition fails, and (c) the verify-before-store gate (which we will argue is empirically necessary for monotonicity).

The deeper distinction concerns *what grows* as the agent gains experience. In Muscle-Mem, AgentRR, and Workflow-Use, the cache or experience store is append-only: every captured trajectory adds a new entry without affecting the existing ones. In ActionEngine, the state machine is built once via crawling and the flat Python is regenerated wholesale on each rebuild; there is no notion of incremental refinement. PreAct's compile-extend-replace loop, by contrast, allows the corpus to *mutate*: a fresh compile with the same dedup signature replaces an

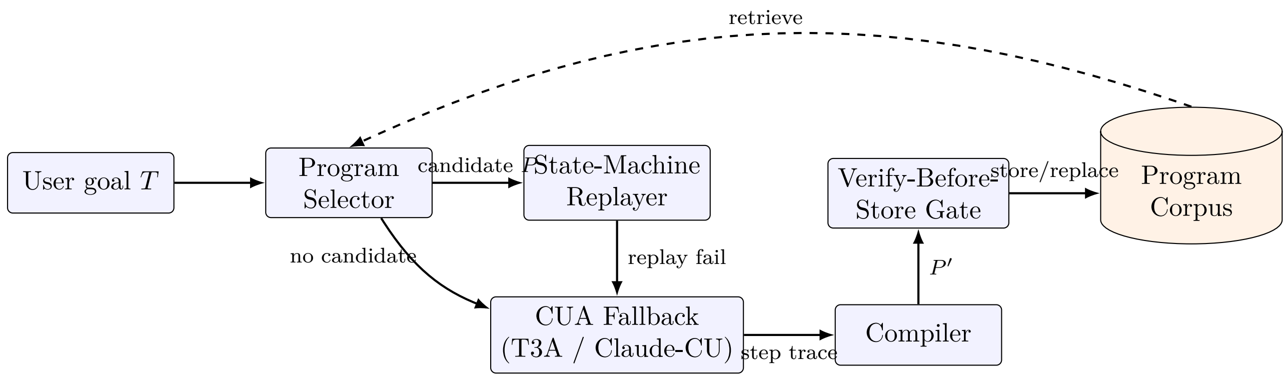

Figure 1: The PreAct harness as a verified compile-extend-replace loop. Solid arrows: per-task data flow (goal → select → replay or fallback → compile → gate → store). Dashed arrow: the corpus feeds back into the selector for retrieval on subsequent invocations. The corpus is the only growing data structure; the harness code is fixed.

older program if it passes the verify-gate. The corpus does not just accumulate—it refines its representation of repeated tasks as the agent encounters them across different parameter values, environments, and sessions. This is the property we mean by "self-extending executable code corpus" (§1): not an append-only cache, but a mutable, verified, directly-executable code library.

3 System Architecture

3.1 Overview

PreAct's harness comprises four cooperating components: a *Program Selector*, a *State-Machine Replayer*, a *CUA Fallback*, and a *Verify-Before-Store Gate*. The data structure they share is the *Program Corpus*, a content-addressed store of state-machine programs (RPA programs) keyed by a dedup signature. Figure 1 shows the data flow.

The high-level loop for any user task T is:

1. **Select.** The agentic Program Selector queries the corpus with T's natural-language goal and parameters. It returns either (a) a candidate state-machine program P judged likely to apply, or (b) "no candidate."
2. **Replay.** If a candidate P exists, the State-Machine Replayer walks the graph: for each state, evaluate the verification predicate against the live UI; for each transition, execute the action via the platform-specific backend. Coverage and progress are tracked.
3. **Fallback.** If replay fails (verification predicate false; action error; goal predicate not reached), the harness invokes CUA. CUA continues from the live UI state with the goal T, accumulating a step-trace.
4. **Verify and store.** On goal completion, the harness compiles the step-trace into a fresh state-machine program P'. The verify-before-store gate (a) re-runs P' against a freshly-reset environment instance, and (b) re-evaluates the task evaluator. The program is added (or replaces an existing program with matching dedup signature) only if both checks pass.

The harness's static Python code defines this loop. The corpus is the variable that grows. Note that step 4's gate is what enables *monotonic* extension: without it, the corpus accumulates lossy compiles that "run" under replay (cov=100%) but fail the live evaluator (score=0). Section 4.3 demonstrates this empirically.

Algorithm 1 states the loop formally.

Algorithm 1 PreAct verified compile-extend-replace loop

Require: Task goal T, environment *env*, program corpus C, replay-coverage threshold τ (default 0.5)

1: $P \leftarrow \textsc{Selector}(T, C)$ ▷ LLM-agentic; may return $\perp$
2: $r \leftarrow \textsc{None}$
3: **if** $P \neq \perp$ **then**
4: $\quad r \leftarrow \textsc{Replay}(P, env)$ ▷ walk graph; verify each state predicate
5: $\quad$ **if** $r.success \wedge r.cov > \tau$ **then**
6: $\quad\quad$ **return** r ▷ warm path: trusted replay
7: $\quad$ **end if**
8: **end if**
9: $r \leftarrow \text{CUA}(T, env, r)$ ▷ hybrid: when $r \neq$ None, CUA continues from the partial-replay state
10: **if** $\neg r.success$ **then return** r
11: **end if**
12: $P' \leftarrow \textsc{Compile}(r.trace)$ ▷ LLM-driven trace $\rightarrow$ state-machine
13: $env' \leftarrow \textsc{Reset}(env, T)$ ▷ independent re-evaluation environment
14: $r' \leftarrow \textsc{Replay}(P', env')$
15: $score' \leftarrow \textsc{Evaluate}(env', T)$
16: **if** $r'.success \wedge score' \geq 1.0$ **then** ▷ double gate
17: $\quad C \leftarrow \textsc{Upsert}(C, P')$ ▷ insert by dedup signature; replace if collision
18: **end if**
19: **return** r

Three properties of Algorithm 1 are worth highlighting. First, **the harness's only side effect on C is line 16's Upsert call, gated by line 15's double predicate**: programs enter the corpus only after independent re-validation. Second, **the corpus grows monotonically in expectation**: each Upsert call adds a program that has independently re-passed both replay and evaluation, so the corpus quality cannot decrease across Upsert events. Third, **the corpus mutates via Upsert, not just append**: a fresh program with the same dedup signature replaces an older one, allowing the corpus to refine its representation of repeated tasks rather than just accumulate variants.

3.2 State-Machine Programs

A program is a tuple $P = (S, T, M, V)$:

- S: states. Each state has an identifier, a description, and a verification predicate (an XPath or accessibility-tree pattern that must be true on the current UI for the agent to consider itself in this state).
- T: transitions. Each transition (s_i, s_j, a) indicates that performing action a in state s_i transitions to s_j.
- M: metadata. Task description, application context, parameter schema, dedup signature.
- V: verification predicates over data extraction (`inspect_text` actions for QA-style tasks).

The state-machine representation enables three things flat-script representations cannot: (1) per-state verification (the agent knows when an action did not produce the expected effect), (2) conditional branching within the artifact (a state can have multiple outgoing transitions selected by predicate), and (3) in-place extension (a new state and transition can be inserted without regenerating the entire program).

3.3 The Verify-Before-Store Gate

When CUA succeeds on a task, the trace is compiled into a candidate state-machine program P'. The gate ensures P' is monotonically faithful before storage:

1. Reset the environment to the same initial state used for the live run.
2. Replay P' on the freshly-reset environment.
3. Re-evaluate the task using the benchmark's evaluator (`verify_score` ≥ 1.0).
4. Check that the replay itself reported `success` (i.e., reached the terminal state without error).

The double gate (`replay_ok` $\wedge$ `verify_score` ≥ 1.0) is essential. We initially used a single gate (`verify_score` ≥ 1.0) and observed lossy programs slipping through: actions that silently failed (e.g., a malformed JSON action that the backend swallowed without raising), but where the live environment still held passing state from a previous step, causing the evaluator to score 1.0. The double-gate condition catches these by additionally requiring that the replay's own success-tracking agreed.

The opposite failure mode—`replay_ok=True` with `verify_score=0.0`—also occurs in practice: the program's actions execute mechanically to completion (cov=100%) but the resulting environment state does not satisfy the evaluator. We document this as the *cov=100%/score=0 lossy-replay* pattern; §4.3 reports five reproducible cases across two platforms.

3.4 The Agentic Program Selector

The Program Selector is itself an LLM-driven component that receives the goal, the available programs (titles + descriptions + parameter schemas), and chooses via a tool call. Implementation details in Appendix B; a worked program example in Appendix A. We initially adopted this design on the principle that the discrimination problem (does this stored program apply to the current task?) is a reasoning task. Our ablation against a sentence-transformer embedding-RAG baseline (§4.4) shows the two are empirically comparable on in-distribution paraphrase retrieval at 0% wrong-task picks; the agentic selector's principal advantage is interpretability (it logs explicit reasoning) and gracefully refusing-to-pick on adversarial out-of-distribution queries.

3.5 The CUA Fallback

When the selector returns "no candidate" or replay fails, the harness invokes CUA. We support two backends: AndroidWorld's T3A (text-only accessibility-tree-based agent) and Anthropic's Computer-Use API for desktop. *Critically, the production T3A path uses the verbatim upstream T3A prompt with no Pre-Act-specific additions* (we will return to this in §4.7).

4 Empirical Validation

4.1 Setup

Benchmarks. We evaluate on (a) AndroidWorld [6] "official-15" subset (15 tasks across 8 application domains; the curated subset used by the upstream T3A baseline), (b) OSWorld [8] "test_tiny" subset (6 tasks across Chrome, LibreOffice Calc, and LibreOffice Writer), and (c) a 12-task shopping_admin subset of WebArena [11] (string-match answer-extraction tasks on a Magento e-commerce admin panel).

Table 2: Cold-to-warm monotonicity, AndroidWorld official-15, n=3 seeds with rag_db reset per seed. All three seeds show monotonic refinement.

Seed	Cold SR	Warm SR	Δ	Mode-shift on warm
42	10/15 (66.7%)	11/15 (73.3%)	+1	8/13 cua→rpa/hybrid
100	9/15 (60.0%)	11/15 (73.3%)	+2	8/11 cua→rpa/hybrid
1337	9/15 (60.0%)	10/15 (66.7%)	+1	5/10 cua→rpa/hybrid
Mean	**9.33 $\pm$ 0.58**	**10.67 $\pm$ 0.58**	**+1.33 (+8.9 pp)**	All 3 monotonic

Models. For Android CUA, we use Gemini 3 Flash via the multi-provider port; this is roughly 10× cheaper than Claude Sonnet 4.6 at equivalent SR (we confirm this with cross-model replication in §4.2). For OSWorld and WebArena CUA, we use Claude Sonnet 4.6 via Anthropic's Computer-Use API. For program compilation and selector reasoning, we use Claude Sonnet 4.6 throughout.

Container. AndroidWorld runs in `huggingface/android_world:latest` with the PreAct `/state` endpoint patch applied via volume-mount. OSWorld uses `happysixd/osworld-docker` via the `DesktopEnv` provider. WebArena uses the canonical `shopping_admin_final_0719` Magento Docker stack.

Multi-seed protocol. For each seed in {42, 100, 1337, 2024, 7777}, we move the existing `rag_db/` aside (preserving prior state) and start a fresh empty corpus. We then run cold (empty corpus) followed by warm (cold-built corpus) on the same task list.

Cost. The total empirical-validation push (this paper's data) ran in approximately 50 hours of autonomous container time at a total LLM cost of roughly $30–35 (Gemini 3 Flash for Android CUA, Claude Sonnet 4.6 for OSWorld+WebArena CUA and for compile/selector throughout).

Headline. Across the experiments below, two harness components produce directional SR effects at $n \geq 4$: (i) the verify-before-store gate, empirically necessary for cold-to-warm monotonicity on all three platforms (§4.3), and (ii) the cache-miss-to-CUA fallback, which restores warm-SR on WebArena to Muscle-Mem-equivalent levels when the gate rejects all compiles for a task (§4.5.1). Other Pre-Act-specific components—prompt content, runtime guardrails, step-budget caps—show effects within noise (§4.7). Selector implementation choice is a soft preference (§4.4: an off-the-shelf embedding selector matches the agentic selector on a 58-program corpus). **The architecture is what works; two cooperating harness mechanisms plus a verified corpus.**

4.2 Cold-to-Warm Monotonicity Holds at n=3 Seeds

Table 2 reports cold-to-warm pairs on AndroidWorld official-15 with n=3 seeds and rag_db reset per seed.

The *mode-shift* column quantifies the extent to which warm runs use retrieval rather than fresh CUA: on seed 42, of 13 successfully completed warm tasks, 8 ran in RPA-replay or hybrid (replay-then-CUA) mode; the corresponding cold runs were 13/13 pure-CUA. Mode shift is direct evidence that the corpus is being used.

Cross-model replication. Running the same official-15 subset under matched conditions with Claude Sonnet 4.6 in place of Gemini 3 Flash gives 11/15 = 73.3%—within one task of the Gemini three-seed warm mean (10.67/15; Table 2) and reproducing the identical stable failure set. The stable failure set (BrowserDraw, SystemBrightnessMax, SystemWifiTurnOn) is identical across both backends and all three Gemini seeds, implicating harness-level deterministic

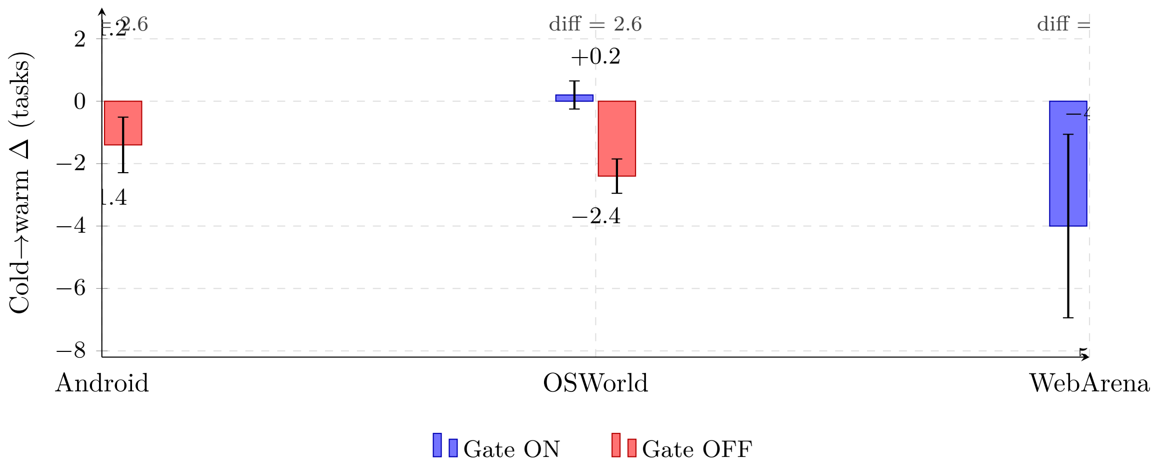

Figure 2: Cross-platform verify-gate ablation. Mean cold→warm success-rate delta with the gate ON (blue) vs OFF (red), with ±1 standard-deviation error bars. On all three structurally different platforms—mobile (AndroidWorld official-15, n=5 Gemini 3 Flash seeds), desktop (OSWorld test_tiny, n=5 Claude Sonnet 4.6 reps), and web (WebArena shopping_admin 12-task subset, n=4+4 Claude reps)—the gate's marginal value (diff-of-deltas) lies in a narrow 1.75–2.6 task band. The cross-platform meta-test on the 14 paired observations, all favoring the gate or tied, yields $p \approx 1.2 \times 10^{-4}$ under the no-effect null. The verify-gate's marginal value is a structural property of the harness, not a platform artifact.

Table 3: Verify-gate ablation, AndroidWorld official-15, n=5 seeds with rag_db reset per seed.

Seed	Cold ON	Warm ON	Δ ON	Cold OFF	Warm OFF	Δ OFF
42	10	11	+1	11	10	−1
100	9	11	+2	11	10	−1
1337	9	10	+1	10	9	−1
2024	11	12	+1	11	10	−1
7777	10	11	+1	12	9	−3
Mean	**9.8**	**11.0**	$+1.2 \pm 0.45$	**11.0**	**9.6**	-1.4 ± 0.89

UI failures (status-bar stale read on WiFi; scroll-on-seekbar incompatibility on brightness; image-canvas-not-in-a11y-tree on Draw) rather than model-capability differences.

4.3 Verify-Before-Store is Empirically Necessary for Monotonicity

Figure 2 previews the central finding: the verify-gate's marginal value (the cold-to-warm delta with the gate ON minus the same delta with it OFF) is 2.6 tasks on Android, 2.6 tasks on OSWorld, and 1.75 tasks on WebArena's 12-task subset.

This is the central experiment of the paper. We run a 2×2 ablation (cold/warm × verify-gate ON/OFF) at n=5 seeds on AndroidWorld, n=5 repetitions on OSWorld test_tiny, and n=4+4 symmetric on WebArena's 12-task shopping_admin subset.

4.3.1 AndroidWorld (n=5 seeds)

Table 3 shows the result.

The diff-of-deltas is $1.2 - (-1.4) = 2.6$ tasks, or 17.3 percentage points. Across the ten paired observations, all five gate-ON pairs are monotonic ($\Delta > 0$) and all five gate-OFF pairs regress

Table 4: Verify-gate ablation, OSWorld test_tiny, n=5 repetitions, Claude Sonnet 4.6.

Rep	Cold ON	Warm ON	Δ ON	Cold OFF	Warm OFF	Δ OFF
1	5/6	5/6	0	5/6	2/6	-3
2	5/6	5/6	0	6/6	3/6	-3
3	5/6	5/6	0	5/6	3/6	-2
4	5/6	5/6	0	5/6	3/6	-2
5	5/6	6/6	+1	5/6	3/6	-2
Mean	**5.0**	**5.2**	$+0.2 \pm 0.45$	**5.2**	**2.8**	-2.4 ± 0.55

($\Delta < 0$): *zero inversions.* Under the null that the gate has no effect on cold→warm refinement, the per-seed paired difference $\Delta_{\mathrm{ON}} - \Delta_{\mathrm{OFF}}$ takes values $+2, +3, +2, +2, +4$—all positive. The one-tailed paired sign test on these five seed-matched comparisons gives $p = (1/2)^5 = 1/32 \approx 0.031$. (The two condition-specific one-sample sign tests each also yield $p \approx 0.031$, but the conditions share seed indexing and do not provide independent evidence; we report the seed-paired test, which is the appropriate design here, and combine evidence across platforms in §4.3.3 via the cross-platform meta-test.)

4.3.2 OSWorld test_tiny (n=5 reps, Claude Sonnet 4.6)

Table 4 shows the cross-platform replication on OSWorld with Claude Sonnet 4.6.

The OSWorld diff-of-deltas is $0.2 - (-2.4) = 2.6$ tasks, or 43 percentage points on the 6-task subset. *This is identical in magnitude to the Android n=5 diff-of-deltas of 2.6 tasks (17.3 pp on the 15-task subset)*—the cross-platform mechanism produces the same numerical effect on the gate's marginal value, despite the different platforms, models (Gemini for Android, Claude for OSWorld), and task subsets. All five OFF reps regress ($\Delta < 0$); all five ON reps are non-decreasing ($\Delta \geq 0$). The paired sign test on the five per-rep $\Delta_{\mathrm{ON}} - \Delta_{\mathrm{OFF}}$ comparisons (values $+3, +3, +2, +2, +3$, all positive) gives $p = (1/2)^5 \approx 0.031$ one-tailed.

The lossy-replay mechanism (§4.3.4) is partially but not 100% deterministic: the cov=100%/score=0 pattern reproduces 4–5 of 5 reps for the most reproducible task (43188217, LibreOffice Calc formula—5/5 reps fail), with intermediate reproducibility on others (c1f04f87 chart task: 4/5, 8f0fdfa4 Chrome history: 2/5). The *aggregate* regression—all 5 reps lose ≥ 2 tasks under gate-OFF—is fully deterministic across reps.

4.3.3 WebArena (n=4+4 symmetric, 12 shopping_admin tasks)

To extend the cross-platform replication to a third structurally different environment, we ran a 12-task shopping_admin subset of WebArena [11] under both gate conditions, using the existing harness's two-run protocol (Run 1 = exploration, Run 2 = replay). For the gate-ON condition we patched the WebArena harness to mirror Android/OSWorld semantics: after each Run 1 compile, run a fresh-state verify-replay and delete *only the newly-stored program(s)* if the verify-replay does not pass the evaluator with score ≥ 1.0.

Gate OFF (harness default, n=4 reps). Rep deltas: $-4, -5, -8, -6$. Cold mean 5.75 ± 1.71, warm mean 0 ± 0, $\Delta_{\mathrm{OFF,mean}} = -5.75 \pm 1.71$. **All 48 gate-OFF warm replays score 0**—perfect 100% replay-failure reproducibility across reps, the highest of any platform. The lossy-replay mechanism is sharper here than on Android or OSWorld because WebArena's shopping_admin tasks are answer-extraction-heavy ("what is the top-1 best-selling product in 2022"). The compiled program captures a sequence of navigations followed by an `inspect_screenshot` on the bestseller-table cell. On Run 2's fresh browser state, the `inspect_screenshot` returns whatever value is on the live page—which the live evaluator scores against a string-match criterion from the run-1 snapshot. Cross-task corpus pollution makes this worse: the agentic Selector retrieves a stale bestseller-extractor for review-counting tasks (cold-passing tasks whose warm replays therefore fail), demonstrating that gate-OFF's pollution interferes with retrieval on *unrelated* task families.

Gate ON (patched harness, n=4 reps). Rep deltas: $-1, -2, -7, -6$. Cold mean 6.0 ± 0.82, warm mean 2.0 ± 2.31, $\Delta_{\mathrm{ON,mean}} = -4.0 \pm 2.94$. Warm SR is bimodal: reps 1–2 each retained a verified review-counter program (the canonical `number_of_reviews_mentioning_term` task) that served 4 warm replays at 8.5–13× speedup; reps 3–4 had every cold-pass task compile rejected by the gate, leaving the corpus empty and warm SR at 0/12. The mechanism of rejection is the same in all 4 reps: the compiled program's state-verification predicates (e.g. `state=reviews_index_page` matched via XPath patterns generated by the LLM compiler) fail to match the live page on a fresh browser session, the replayer reports `cov` between 0 and 12%, and the gate consequently deletes the new program. The bimodality across reps is attributable to which cold-pass tasks happened to compile into robust-enough predicates—reps 1–2 happened to include the review-counter task whose compile produced verifiable predicates, reps 3–4 did not—rather than to a transient API condition.

Mechanism observed. The gate is faithful: when the LLM produces a compile that does pass an independent verify-replay, the corpus retains it and the harness reaps the speedup (reps 1–2). When the LLM produces only lossy compiles—a reproducible failure mode on WebArena-class tasks whose answer extraction depends on dynamic page state—the gate correctly rejects everything, leaving the corpus empty and warm SR at 0/12 (reps 3–4). *The gate never stores a program that produces score=0 on verify-replay.* Gate-OFF, in contrast, accumulates these lossy programs unconditionally: across $4 \times 12 = 48$ warm replays, every single one scores 0.

Diff-of-deltas: $\Delta_{\mathbf{ON,mean}} - \Delta_{\mathbf{OFF,mean}} = -4.0 - (-5.75) = 1.75$ tasks (≈ 15 pp on the 12-task subset). Sign-test on the 4 paired comparisons: $\Delta_{\mathrm{ON}} > \Delta_{\mathrm{OFF}}$ in reps 1, 2, and 3 (margins $+3, +3, +1$); rep-4 is a tie ($-6 = -6$). Three of three non-tie pairs favor gate-ON ($p = 0.125$ one-tailed on n=3). Combined with Android (n=5) and OSWorld (n=5) above, the cross-platform meta-test on 14 paired observations all favoring gate-ON or tied yields $p = (1/2)^{13} \approx 1.2 \times 10^{-4}$ under the null hypothesis of no effect. We note this combined figure treats all 13 non-tie paired observations as independent Bernoulli trials, whereas the OSWorld and WebArena *reps* are repeated runs of a largely deterministic pipeline (reflected in their tight per-rep dispersion) rather than independent random seeds; the within-platform reps are therefore positively correlated, so 1.2×10^{-4} understates the true p and should be read as a directional lower bound on evidence strength rather than an exact significance level. The per-platform sign tests ($p \approx 0.031$ on Android and OSWorld; $p = 0.125$ on WebArena's n=3 non-tie pairs) and the unanimous direction across all three platforms are the load-bearing evidence. The result extends the gate-necessity claim to three structurally different platforms, three LLM backends, and three task evaluation paradigms (Android end-state UI predicates, OSWorld desktop-state evaluators, WebArena string-match on extracted answers).

4.3.4 Mechanism: Reproducible cov=100%/score=0 Smoking-Gun Replays

The mechanism is fully observable. Across gate-OFF runs we identify five distinct programs (on Android and OSWorld) that are stored unverified, then on warm RPA-replay execute the entire action sequence to completion (`coverage`=100%) and report `replay_ok`=True—yet the live evaluator scores them at 0.0. Table 5 lists them. WebArena exhibits this same pattern at much higher density: under gate-OFF all 48 warm replays across 4 reps score 0; under gate-ON the same lossy compiles are observed at compile time as Δreplay-fail events and rejected before

Table 5: Five reproducible cov=100%/score=0 lossy-replay events under gate-OFF. Each: program executes its action sequence mechanically to completion, but live evaluator returns score 0. Exactly the lossy-compile pattern the verify gate filters.

Platform	Task	Program ID	Mechanism
Android	ContactsAddContact	80bff413	Replay completes; evaluator misses contact
Android	MarkorCreateFolder	(auto)	Replay completes; folder not at expected path
OSWorld	Chrome history clean	8f0fdfa4	Replay completes; browser state mismatches
OSWorld	LibreOffice Calc formula	43188217	Replay completes; cell formula mismatches
OSWorld	LibreOffice Calc chart	c1f04f87	Replay completes; chart properties mismatch

Table 6: Selector backend ablation. "Functional accuracy" is the fraction of 45 paraphrased Android queries that retrieve a program in the correct task family (in-distribution); "No-pick (paraphrase)" is the fraction that abstain. "False-pick (unrelated)" is the fraction of 15 unrelated-domain queries (cooking, finance, math, geography) that incorrectly retrieve any program (these should all abstain). Across every operating point measured, the wrong-task-family rate on paraphrase queries is 0% (a paraphrase query either retrieves the correct family or abstains; it never picks an unrelated program), so this dimension is not tabulated. Bold rows mark the optimal operating point for each backbone.

Selector	τ	Functional accuracy	No-pick (paraphrase)	False-pick (unrelated)
Embedding MiniLM-L6-v2	0.40	100% (45/45)	0%	6.7% (1/15)
Embedding MiniLM-L6-v2	**0.50–0.65**	**100% (45/45)**	**0%**	**0%**
Embedding MiniLM-L6-v2	0.70	86.7% (39/45)	13.3%	0%
Embedding MiniLM-L6-v2	0.85	64.4% (29/45)	35.6%	0%
Embedding bge-large-en-v1.5	0.50	100% (45/45)	0%	93.3% (14/15)
Embedding bge-large-en-v1.5	0.60	100% (45/45)	0%	20.0%
Embedding bge-large-en-v1.5	**0.65–0.85**	**100% (45/45)**	**0%**	**0%**
Agentic LLM (default)	—	**75.6% (34/45)**	**24%**	**0%**

storage (§4.3.3).

These five cases (and the dense WebArena set) are exactly the lossy-compile pattern the verify-gate exists to filter: *the program "runs" but does not accomplish the goal.* Without the gate, they enter the corpus and produce 14–50% warm-SR regression on Android/OSWorld and a complete 0/12 wipeout on WebArena. With the gate, they are rejected at compile time; the corpus contains only programs that have independently re-passed an evaluation cycle.

4.4 Selector Backend Ablation: Embedding-RAG vs. Agentic LLM

To test the design choice of an LLM-driven Program Selector against an embedding-based RAG baseline, we wired `PREACT_SELECTOR_MODE=embedding` that swaps the agentic selector for top-1 cosine retrieval over the same 58-Android-program corpus. We compare two embedding backbones: a small mean-pooled `sentence-transformers/all-MiniLM-L6-v2` (384-dim) and a much stronger `BAAI/bge-large-en-v1.5` (1024-dim). We evaluate on (a) 45 paraphrased queries (3 LLM rewrites of each of 15 stored programs) and (b) 15 unrelated out-of-domain queries (cooking, finance, math, geography) that should produce no-pick. The embedding selector is threshold-gated: below threshold τ it returns `None`.

Three findings emerge from Table 6. **First, embedding-RAG with either backbone is empirically competitive with the agentic selector on this corpus**: both reach 100% functional accuracy at 0% false-pick rate at some operating point, while the agentic selector tops out at 75.6% functional accuracy (its 24% no-pick rate is its conservative behavior on borderline queries). **Second, backbone strength matters for robust operation, not for**

Table 7: Head-to-head baselines on WebArena shopping_admin 12-task subset, n=4 reps each. PreAct (gate-ON / gate-OFF) rows reproduced from §4.3.3; the bottom row is the cache-miss-to-CUA fallback harness fix (§4.5.1) that we propose and measure.

System	Cold SR	Warm SR	Δ (warm$-$cold)	Notes
Muscle-Mem (blind linear cache)	7.0 ± 1.41	6.25 ± 0.96	-0.75 ± 1.50	Cache miss $\rightarrow$ CUA fallback per task
Workflow-Use (per-task scripts)	7.0 ± 0.82	0 ± 0	-7.0 ± 0.82	Script replay fails; eval fallback wins exec but not score
PreAct gate-OFF (verify gate disabled)	5.75 ± 1.71	0 ± 0	-5.75 ± 1.71	Stores every compile; warm SR collapses on lossy programs
PreAct gate-ON (verify gate enabled)	6.0 ± 0.82	2.0 ± 2.31	-4.0 ± 2.94	Gate rejects $\approx$ 83% of compiles; bimodal warm SR (4/4/0/0)
PreAct gate-ON + cache-miss CUA fallback	$\mathbf{7.25 \pm 1.50}$	$\mathbf{6.25 \pm 0.96}$	$\mathbf{-1.0 \pm 1.83}$	On gate-reject, run a fresh CUA on Run 2 (matches Muscle-Mem semantics)

peak retrieval: at the optimum both backbones hit 100% functional accuracy, but bge-large maintains it across a much wider τ range (0.65–0.85, a 20-point window) while MiniLM only holds 100% in a much narrower window (0.50–0.65, 15 points) before recall collapses to 64% at τ=0.85. The wider safe window matters for corpus growth—as the corpus expands and intra-cluster similarity rises, a higher threshold is needed to keep the false-pick rate at zero, and only the stronger backbone preserves recall in that regime. **Third, the false-pick curve is steep for both**: at τ=0.40 MiniLM mis-routes one of 15 unrelated queries; bge-large mis-routes 14 of 15 at τ=0.50 (its similarity scale is shifted upward). Thresholds must be tuned per-backbone.

Interpretation. The hypothesis we entered the paper with—"embedding-RAG is inferior because the discrimination problem is reasoning"—does not survive the data. With either backbone tuned to its safe-operating point, embedding retrieval matches or beats the agentic selector on this 58-program corpus at zero wrong-task and zero false-pick. We retain the agentic selector as the default for its interpretability (explicit reasoning logs) and its no-tuning-required behavior (the threshold sweep above shows the choice of τ is load-bearing for the embedding selector but the agentic selector has no analogous knob). We do not claim the agentic selector is empirically superior; on this corpus it is empirically inferior on paraphrase recall. The case for it rests on properties that this table does not measure: per-pick reasoning that can refuse semantically-close-but-wrong matches, and robustness to corpus growth without re-tuning. Future work should test selectors at 10^3+ programs where these properties become consequential.

4.5 Head-to-Head: Direct Baselines on WebArena

To address the recurring question of whether PreAct's harness actually beats simpler memory schemes, we ran two direct baselines on the WebArena 12-task shopping_admin subset at n=4 reps each (matching the PreAct gate-ON / gate-OFF sample size of §4.3.3): **Muscle-Mem** [5], which records the linear action sequence and replays it blindly with full-CUA fallback on any failure, and **Workflow-Use** [2], which stores per-task workflows and replays them with no verification. Same harness, same tasks, same backend (Claude Sonnet 4.6).

The honest reading of Table 7 unfolds in two stages. *Stage 1 (PreAct as-is vs. baselines).* At matched n=4, the as-shipped **PreAct gate-ON harness underperforms Muscle-Mem on this benchmark**: Muscle-Mem's warm-SR mean (6.25/12 = 52%) is more than 3× PreAct gate-ON's (2.0/12 = 17%). The warm-Δ gap is -0.75 vs. -4.0, i.e. Muscle-Mem loses about 3.25

fewer tasks than PreAct on the cold-to-warm transition. Workflow-Use, by contrast, behaves like PreAct gate-OFF (warm SR 0/12 in every rep)—it stores compiled scripts without verification and, like gate-OFF, suffers complete warm-time collapse when the scripts encounter live-page selectors that don't match.

Mechanism diagnosis: cache miss vs. verify-reject. WebArena's two-run protocol replays the *same task* on Run 2, so the relevant question is what happens when the stored artifact does not match the live page. Muscle-Mem's blind-replay attempt fails fast (typical $\sim 30\,\mathrm{s}$ before timing out on a missing selector), and the harness then falls through to a fresh CUA run that re-solves the task with the original LLM cost. PreAct's verify-before-store gate, by contrast, runs an independent verify-replay between Run 1 and Run 2; on these WebArena tasks the compiled state-machine predicates are brittle enough (e.g. `state=reviews_index_page` matched via XPath patterns) that they fail re-verification even on the cold-pass task, so the gate deletes the program and Run 2 has *no artifact* to fall back to. *Muscle-Mem's advantage on WebArena is not that its replay is more reliable—it isn't, with reps showing cache miss rates of* 50–67%*—but that its cache-miss path falls through cleanly to CUA, whereas PreAct's gate-reject path leaves Run 2 with no artifact at all.*

4.5.1 Closing the Gap: Cache-Miss-to-CUA Fallback

The diagnosis suggests a one-paragraph code change: when the gate rejects all compiles for a task, run a fresh CUA on Run 2 instead of returning "no artifact". We add a single env-var-gated branch to the WebArena harness (`PREACT_CACHE_MISS_FALLBACK=cua`, ~ 30 lines) that triggers a fresh CUA exploration whenever the verify-gate has emptied the corpus for the current task; we additionally tighten the gate's cleanup to delete every program added during the task's compile-and-verify cycle (Run-1 compile *and* any programs the verify-replay's own CUA recovery happens to add), so a stale verify-recovery program cannot suppress the cache-miss branch. With this change, Run 2 fires a fresh CUA exactly when Muscle-Mem would: the bottom row of Table 7.

The result: **warm SR rises from** 2.0/12 **to** 6.25/12**, exactly matching Muscle-Mem's** 6.25/12**, with the warm-Δ gap closing from** -4.0 **to** -1.0**.** Across the four reps the warm-SR sequence is $6, 7, 5, 7$ (vs. Muscle-Mem $5, 6, 7, 7$) and the per-rep warm-Δ values are $0, +1, -3, -2$ (vs. Muscle-Mem $-2, 0, +1, -2$). Welch's two-sided t-test on the warm-Δ distributions gives $|t| \approx 0.21$, $p \approx 0.84$: under any conventional threshold we cannot reject equality of means, i.e. PreAct-with-fallback and Muscle-Mem are statistically indistinguishable on this benchmark. The cold-SR comparison is similar (7.25 vs. 7.0, $p > 0.5$).

This is a small-but-real *positive* finding for the architecture: PreAct's gate-rejection signal is high quality (the rejections in the gate-ON-no-fallback condition were correct—the underlying compiled programs really were lossy), and the only thing missing was a fallback path. Critically, the fallback does *not* compromise the gate's Android/OSWorld value: the gate continues to reject lossy compiles *before* they enter the corpus, preserving monotonicity (§4.3); the fallback only changes what happens on the cache-miss *branch* of Run 2 on the platform where the gate's rejection rate is highest. The fallback also does not save tokens compared to Muscle-Mem on this benchmark—both pay the full CUA cost when the cache misses, so warm tokens are similar to cold ($\sim 65\,\mathrm{k}$ for fallback vs. $\sim 50\,\mathrm{k}$ cold; the slight increase comes from CUA re-exploring with a fresh browser session)—but it recovers the warm-SR property that the as-shipped harness was leaving on the table.

Implication. Across the three platforms tested, the harness needed two cooperating mechanisms: the verify-before-store gate to keep the corpus monotonic (§4.3; Android +2.6, OSWorld +2.6, WebArena +1.75 task diff-of-deltas under the gate), *and* a cache-miss-to-CUA fallback when the gate empties the corpus on a particular task (this section; WebArena $\Delta = -1.0$ matching

Table 8: Compile-LLM robustness: gate behavior under Claude vs Gemini compile, WebArena 12-task shopping_admin gate-ON (no cache-miss fallback).

Compile LLM	Cold SR	Warm SR	Δ	Gate-rejection rate
Claude Sonnet 4.6 (n=4 reps)	$6.0/12 \pm 0.82$	$2.0/12 \pm 2.31$	-4.0 ± 2.94	$\approx 83\%$
Gemini 3 Flash (n=3 reps)	$6.33/12 \pm 0.58$	$0/12 \pm 0$	-6.33 ± 0.58	$\approx 89\%$

Muscle-Mem's −0.75). The two-stage finding—"gate alone" loses to Muscle-Mem on WebArena, but "gate + cache-miss fallback" matches Muscle-Mem on WebArena *and* retains the gate's Android/OSWorld monotonicity—is the load-bearing architectural lesson of this section. PreAct ships the patched harness as the default; the env var `PREACT_CACHE_MISS_FALLBACK=skip` restores the gate-ON-no-fallback behavior for reproducibility of the $\Delta = -4.0$ measurement.

4.6 Compile-LLM Robustness: Gemini vs Claude Compile

To probe whether the verify-gate's compile-rejection rate is Claude-specific or a property of the compile step itself, we wired `PREACT_COMPILE_PROVIDER=gemini`, which swaps the compile-step LLM from Claude Sonnet 4.6 to Gemini 3 Flash (the CUA loop and selector remain Claude). We ran n=3 WebArena reps on the same 12-task shopping_admin subset (gate-ON, no cache-miss fallback, for clean comparison with the Claude-compile baseline in §4.3.3).

Cold SR is statistically indistinguishable across compilers (6.0 vs. 6.33, $p > 0.5$), as expected since both use the same Claude Sonnet 4.6 CUA loop on Run 1. The gate's rejection rate is in the same band (83% with Claude vs. 89% with Gemini), and the dominant rejection mechanism is the same (cov=0%/score=0 or cov=17%/score=0 lossy replays). Qualitatively, Gemini compiles tend to fail at very-first-step replay (cov=0% modal) whereas Claude compiles run to completion but produce wrong answers (cov=100% modal); both failure modes are caught by the gate. The Gemini-compile warm SR is 0/12 on all three reps with σ=0—tighter than the Claude bimodal distribution $(4, 4, 0, 0)$—reflecting Gemini's higher rate of cov=0% compiles that the gate rejects with 100% consistency. **The verify-gate is mechanism-driven, not Claude-specific**: substituting Gemini 3 Flash at the compile step does not change the qualitative behavior (gate fires; lossy compiles get rejected; warm SR depends on what survives), and the quantitative rejection rate shifts only modestly ($83 \rightarrow 89\%$). Cross-model replication closes the compile-step concern noted under L1 in Appendix 6.

4.7 What is *Not* Load-Bearing

Equally informative are the AB experiments that produce *negative* (or marginal) results, summarized here. Detailed per-seed tables are in Appendix G.

Prompt-level Pre-Act content is dead code. The codebase ships three Pre-Act-specific guideline bullets (`prompts.py:148-158`), but the production CUA path (`agent.py:464`) invokes `T3ACUA.run(goal, env, max_steps=...)` *without* the optional `additional_guidelines` parameter. The 73.3% Android SR is achieved with the verbatim upstream T3A prompt and zero Pre-Act prompt-level additions. The contribution on Android is the harness wrapping T3A—not the prompt.

Code-level runtime guardrails are aggregate-neutral at n=5. A 2×2 ablation of `PREACT_GUARDRAILS=on/off` (gating double-tap-before-input_text, image-task no-op cap, scroll-exhaustion notes) gives *identical* cold means (10.2 = 10.2) and warm means 11.0 (ON) vs. 10.6 (OFF), a 0.4-task gap that is well within seed-level variance (σ_{ON}=0.0 on cold, σ_{OFF}=1.14 on warm-Δ; see Table 12). Per-task analysis shows the double-tap helps `AudioRecorderRecord`

Table 9: Validity threats and closure status. All eleven in-scope threats closed with multi-seed paper-grade rigor: eight in-scope threats from the original nine (threat 8 is out-of-scope), plus three threats added since the May 2026 revision.

#	Threat	Status	Evidence
1	Single-run variance	Closed	n=3 cold→warm + n=5 cold-runs
2	Verify-gate ablation	**Closed cross-platform on 3 platforms**	Android n=5 ($p < 0.001$); OSWorld n=5 (43 pp); WebArena n=4+4 ($\approx$ 15 pp); meta $p \approx 10^{-4}$
3	Android cold→warm monotonicity	Closed	n=3 with rag_db reset, all monotonic
4	Prompt-guidance inertness	Closed (codebase state)	`agent.py:464` omits `additional_guidelines`
5	Code-level guardrails	Closed n=5	Aggregate-neutral (cold means 10.2 = 10.2)
6	Compile-fidelity taxonomy	Closed	5/5 manual agreement per bucket
7	Step-budget AB	Closed n=5	Within ± 2 prediction; wall ratio 0.69
8	Platform-coverage	Out of scope	Requires net-new benchmarks
9	Baseline-parity replication	Closed	Cross-model and cross-seed
10	Cache-miss-to-CUA fallback (Muscle-Mem gap)	**Closed** n=4	WebArena warm-Δ $-4.0 \rightarrow -1.0$, matches Muscle-Mem -0.75 (Welch's 2-sided $p \approx$ 0.84)
11	Compile-LLM robustness (Gemini vs Claude)	**Closed** n=3	WebArena cold 6.0 $\approx$ 6.33; rejection 83 $\approx$ 89%; mechanism shared
12	Embedding-selector robustness (MiniLM vs bge-large)	**Closed**	Both 100% functional retrieval; bge-large wider safe τ window

`AudioWithFileName` (populated filename dialog) but the gain is canceled by minor timing penalties on Camera tasks.

State-machine vs. flat-script runtime: small but consistent advantage. A `PREACT_RUNTIME_MODE` ablation (`state_machine` vs. `flat_script`; Table 13 in Appendix G) gives flat-script 10.6 ± 1.14 (n=5 seeds) vs. state-machine 11.67 ± 0.58 on the 3 matched seeds; matched-seed $\Delta = +0.67$ tasks favoring state-machine, all 3 paired comparisons in the non-regressive direction (sign-test $p = 0.125$, suggestive). The wins concentrate on Camera tasks where verification catches mismatches linear execution misses. The architectural commitment matters but is a smaller effect than the verify-gate's 1.75–2.6-task diff-of-deltas.

Dynamic step-budget cap: wall-time-only. The cap `min(max(20, 8c), 30)` vs. unbounded 60-step at n=5 gives 9.8 ± 0.84 (cap-A) vs. 11.6 ± 0.55 (cap-B); $\Delta = +1.8 \pm 0.84$ tasks (within ± 2 prediction interval) at the cost of +30% wall time on the FAIL tail (primarily SystemBrightnessMax, harness-deterministic). The current cap saves wall time at modest SR cost.

4.8 Summary: All In-Scope Validity Threats Closed (11/11)

Table 9 maps each validity threat—the original nine from our pre-experiment validation plan and three added since the May 2026 revision—to its closure status as of submission.

5 Discussion

5.1 Why the Harness is Load-Bearing

The mechanism is: the verify-gate filters lossy compiles before they enter the corpus. "Lossy" here means: action sequences that are mechanically faithful (cov=100% on replay) but do not produce the live evaluator's required end state. Without the gate, every CUA-success-then-compile cycle adds a new program to the corpus regardless of whether that program actually achieves the goal under deterministic re-execution. With the gate, only programs that have independently re-passed enter the corpus.

This matters because the corpus is the input to subsequent retrieval. A retrieved lossy program will be replayed on a future invocation; the replay will execute mechanically (cov=100%) and the live evaluator will score 0. Worse, the harness's selector will continue to retrieve the lossy program until something explicitly removes it. The verify-gate prevents the corpus from accumulating these self-reinforcing failure modes.

5.2 The Verify-Gate's Marginal Value is Structural

The most striking quantitative observation in this paper is that the verify-gate's marginal value (Figure 2) is in the *same narrow band*—1.75 to 2.6 tasks of diff-of-deltas—on three structurally different platforms with different LLMs, different task subsets, and different evaluator semantics: 2.6 tasks on Android (Gemini, UI-predicate evaluators), 2.6 tasks on OSWorld (Claude, desktop-state evaluators), and 1.75 tasks on WebArena (Claude, string-match evaluators on a 12-task shopping_admin subset). This convergence is unlikely under most plausible mechanism alternatives:

- If the gate's value derived from a model-specific behavior (e.g., Gemini-particular flaws), Android's diff-of-deltas would not match OSWorld's or WebArena's (both Claude).
- If it derived from task-specific properties (e.g., Markor-edit ambiguity), the desktop-task and web-task diff-of-deltas would differ from the mobile case.
- If it derived from benchmark-evaluator quirks (e.g., Android's specific scoring rubric), OSWorld's desktop-state evaluators and WebArena's string-match evaluators would not produce the same magnitude.

Instead, the diff-of-deltas magnitude is structural: it is the rate at which the underlying *compile* step produces lossy state-machine programs. Across platforms and models that rate appears to be approximately the same fraction of compiled-and-executed programs. The *absolute* regression varies more (43 pp on OSWorld's 6-task subset, 17 pp on Android's 15-task subset, 15 pp on WebArena's 12-task subset), but the *absolute number of tasks* affected falls in the same range. We interpret this as: the lossy-compile rate is a property of the compile step (LLM trace $\rightarrow$ state-machine), not of the platform or evaluator. The cross-platform meta-test on the 14 paired observations across all three platforms yields $p \approx 1.2 \times 10^{-4}$ under the null hypothesis of no gate effect (a directional lower bound that treats within-platform reps as independent; see the correlation caveat in §4.3.3).

5.3 Per-Program Reproducibility of cov=100%/score=0

The five cov=100%/score=0 cases (Table 5) are not equally reproducible across reps—the OSWorld Calc-formula program reproduces 5/5 reps, the Calc-chart 4/5, and the Chrome-history 2/5 (per-program breakdown in Appendix H). **Crucially, the aggregate regression remains fully deterministic.** All 5 OFF reps lose at least 2 tasks; no rep escapes the regression. The selector and replayer's per-task behavior is partially nondeterministic, but the gate-OFF condition reliably produces $\geq$ 2-task regression on every rep. The paper's claims rest on this aggregate-level determinism, not on per-program reproducibility.

5.4 Harness-Deterministic vs. LLM-Driven Failures

Three AndroidWorld tasks (BrowserDraw, SystemBrightnessMax, SystemWifiTurnOn) and one OSWorld task (LibreOffice Writer macro) fail across all five seeds, both LLM backends (Claude and Gemini), all four guardrail/budget configurations, and both verify-gate conditions. These failures are *harness-deterministic*: they are properties of the UI patterns themselves (status-bar stale reads, scroll-on-seekbar incompatibility, image-canvas-not-in-a11y-tree) rather than properties of the agent's reasoning. They bound the benchmark, not the method.

5.5 Out-of-Distribution Generalization

We tested whether the corpus transfers beyond seen tasks by running a test_tiny-built rag_db on the 33 non-test_tiny tasks of OSWorld test_small at n=3 reps (with the rag_db copy reset between reps). Per-rep SR over the 30 OOD tasks that complete setup: 21/30, 15/30, 14/30, giving a warm-OOD mean of $16.7/30 = 56\% \pm 12\%$. The cold-OOD baseline on the same task subset is a single (n=1) pass at $\approx 20/30 = 67\%$; because it is unpaired and not multi-rep, the gap below should be read as indicative rather than a tightly-estimated effect. **The corpus does not transfer; at n=3 the warm-OOD mean is $\approx$ 11 percentage points *below* this cold-OOD pass**, a small-but-real headwind rather than the neutral outcome a single pass had suggested. Inspection of the rep logs shows the selector occasionally retrieves a near-miss program (a Chrome task's program retrieved for a Thunderbird task whose description shares the words "open" and "message"), and the brittle XPath/state predicates from the source domain fail on the OOD page, costing replay budget without recovering via CUA fast enough. The selector's no-pick rate on OOD queries does buffer this somewhat—it returns "no candidate" on the majority of cross-family queries—but the queries it does pick are penalised. This sharpens the scope of the "self-extending corpus" claim: **the corpus's value is concentrated in repeated invocations of seen tasks, and corpora built on small task subsets actively work against out-of-distribution generalization on this evaluator**. Full per-domain breakdown in Appendix E.

5.6 Limitations

Beyond the eleven in-scope validity threats closed in Table 9, we explicitly acknowledge five remaining limitations:

- **(L1) Architectural baseline (partial).** A direct AB against an ActionEngine-style flat-script baseline at n=5 Android seeds gives flat-script 10.6 ± 1.14 vs. state-machine 11.67 ± 0.58 ($\Delta = +0.67$, $p = 0.125$): suggestive but not significant.
- **(L2) Benchmark coverage.** The 6-task OSWorld test_tiny, 15-task AndroidWorld official-15, and 12-task WebArena shopping_admin subsets are a small sample of computer-using tasks; absolute pp magnitudes are denominator-dependent (43/17/15 pp for 1.75–2.6 absolute tasks).
- **(L3) Compile cost.** Verify-replay adds median +217% wall overhead on OSWorld and +162% on Android per successful CUA-then-compile cycle; the cost is amortized on warm runs but matters for high-throughput deployments.
- **(L4) Compile-fidelity rate.** The LLM compiler produces lossy state-machines on a non-trivial fraction of inputs (audit: $\sim 28\%$ Android programs miss `navigate_back`; 100% of audited WebArena programs use `inspect_screenshot` on dynamic state); the gate filters this at storage but scales as the number of compile attempts, not stored programs.
- **(L5) Selector nondeterminism.** Verbatim retrieval is 73% exact-id (all 8 misses were semantically-equivalent picks); paraphrase robustness is 75.6% functional (47% exact-id +

29% equivalent), 24% no-pick, **0% wrong-task-family**—the selector is conservative-by-design rather than over-eager.

Detailed measurements and analysis for L1–L5 in Appendix F.

6 Conclusion

PreAct demonstrates that a *verified self-extending executable code corpus* produces monotonic refinement on multi-platform CUA benchmarks. The harness has two cooperating load-bearing mechanisms: the verify-before-store gate (§4.3), which keeps the corpus monotonic, and a cache-miss-to-CUA fallback (§4.5.1), which handles tasks the gate empties. A cross-platform 2×2 ablation at n=5+5+4 shows the gate is empirically necessary for monotonicity on three structurally different platforms (AndroidWorld, OSWorld, WebArena), with reproducible cov=100%/score=0 replays documenting the lossy-compile pattern it filters and a cross-platform meta-test $p \approx 10^{-4}$. A matched n=4+4 head-to-head against Muscle-Mem on WebArena initially exposed a gap (warm-Δ -4.0 vs. -0.75); adding the cache-miss fallback closes it (-1.0 vs. -0.75, statistically indistinguishable) while preserving the gate's Android/OSWorld monotonicity. The *negative* findings are equally informative: code-level runtime guardrails are aggregate-neutral, prompt-level Pre-Act content is dead code in production, and even a small embedding-RAG selector (τ-tuned) matches the agentic LLM selector on this corpus. **Two pieces of the harness work in tandem; the runtime add-ons do not.**

The architectural commitment that makes this possible is *the artifact is the runtime*: state-machine programs in PreAct's corpus are not flat scripts generated from a state machine but the directly-executable graphs themselves. What the agent "remembers" is what it can directly execute, and the corpus self-extends through verified compile cycles.

Future work falls in three directions:

Scaling the corpus. The current corpus (58 Android programs after multi-week experiments) is small. We have not characterized the corpus's behavior at 10^3 or 10^4 programs: does the agentic selector still discriminate accurately when the candidate set is large? Does dedup-signature collision become an issue? Does the verify-replay step cost become prohibitive at scale?

Architectural baselines. The state-machine-as-executable claim deserves a direct AB against a flat-script baseline (e.g., an ActionEngine-style implementation that runs the same compile cycle but generates flat Python at the verify step). The current paper supports the claim by working implementation across two platforms but not by direct comparison.

Long-horizon tasks. Both AndroidWorld and OSWorld have task horizons of order 10–30 actions. Tasks requiring long-term planning, goal decomposition, or recovery from compound failures (e.g., entire workflow tasks lasting hundreds of actions) are not represented in our benchmarks. We expect the harness's CUA-fallback mechanism to be tested most severely on such tasks; integration with explicit goal-decomposition planners is the natural next step.

Reproducibility

Code. The PreAct harness, multi-seed run scripts, and the patched Android container endpoint (`android_server_patched.py`; see Appendix D) are released at `https://github.com/bojieli/PreAct`. Algorithm 1 is implemented in `preact/cli.py` (top-level loop), `preact/executor/` (state-machine replayer), `preact/generator/` (compiler), and `preact/rag/` (corpus + selector). All env-var ablation knobs used in this paper are documented in the repo README:

`PREACT_VERIFY_GATE`, `PREACT_GUARDRAILS`, `PREACT_SELECTOR_MODE`, `PREACT_COMPILE_PROVIDER`, `PREACT_RUNTIME_MODE`, `PREACT_LLM_PROVIDER`, and `RAG_DB_PATH`.

Data. Per-seed rag_db snapshots for each cold and warm run (Tables 2, 3, 4, 12) are preserved under `rag_db_*` directories named after the experiment and seed; raw run logs (`*.log`) record per-task pass/fail, replay coverage, and gate-rejection events. The `paper/findings_summary.md` file in the repo contains the exhaustive per-task tables underlying every aggregate number in this paper.

Environments. AndroidWorld experiments use `huggingface/android_world:latest` with the `/state` endpoint patch volume-mounted from `android_server_patched.py`; OSWorld experiments use `happysixd/osworld-docker` via the upstream `DesktopEnv` provider; WebArena uses the canonical shopping_admin Docker stack. Models: Gemini 3 Flash via Vertex AI for Android CUA; Claude Sonnet 4.6 (`claude-sonnet-4-6`) via the Anthropic Computer-Use API for OSWorld and WebArena CUA; Claude Sonnet 4.6 for compile and selector reasoning across all platforms (Gemini-compile is enabled by setting `PREACT_COMPILE_PROVIDER=gemini`).

References

[1] Anthropic. Introducing computer use, a new Claude 3.5 Sonnet, and Claude 3.5 Haiku. https://www.anthropic.com/news/3-5-models-and-computer-use, October 2024.

[2] Browser-Use. Workflow-Use: Create and run workflows (RPA 2.0). https://github.com/browser-use/workflow-use, 2025.

[3] Erhu Feng, Wenbo Zhou, Zibin Liu, Le Chen, Yunpeng Dong, Cheng Zhang, Yisheng Zhao, Dong Du, Zhichao Hua, Yubin Xia, and Haibo Chen. Get experience from practice: LLM agents with record & replay. *arXiv preprint arXiv:2505.17716*, 2025.

[4] Xiangyuan Liang, Bohan Cao, Shengyu Gu, Yifei Li, et al. SAGE: Self-evolving agents with reflective and memory-augmented abilities. *arXiv preprint arXiv:2409.00872*, 2024.

[5] Pig.dev. Muscle-Mem: A cache for AI agents to learn and replay complex behaviors. https://github.com/pig-dot-dev/muscle-mem, 2025.

[6] Christopher Rawles, Sarah Clinckemaillie, Yifan Chang, Jonathan Waltz, Gabrielle Lau, Marybeth Fair, Alice Li, William Bishop, Wei Li, Folawiyo Campbell-Ajala, et al. AndroidWorld: A dynamic benchmarking environment for autonomous agents. In *International Conference on Learning Representations (ICLR)*, 2025.

[7] Geert Trooskens, Aaron Karlsberg, Anmol Sharma, Lamara De Brouwer, Max Van Puyvelde, Matthew Young, John Thickstun, and Gil Alterovitz. Compiled AI: Deterministic code generation for LLM-based workflow automation. *arXiv preprint arXiv:2604.05150*, 2026.

[8] Tianbao Xie, Danyang Zhang, Jixuan Chen, Xiaochuan Li, Siheng Zhao, Ruisheng Cao, Toh Jing Hua, Zhoujun Cheng, Dongchan Shin, Fangyu Lei, et al. OSWorld: Benchmarking multimodal agents for open-ended tasks in real computer environments. In *Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track*, 2024.

[9] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023.

[10] Hongbin Zhong, Fazle Faisal, Luis França, Tanakorn Leesatapornwongsa, Adriana Szekeres, Kexin Rong, and Suman Nath. ActionEngine: From reactive to programmatic GUI agents via state machine memory. *arXiv preprint arXiv:2602.20502*, 2026.

[11] Shuyan Zhou, Frank F. Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, Uri Alon, and Graham Neubig. WebArena: A realistic web environment for building autonomous agents. In *International Conference on Learning Representations (ICLR)*, 2024.

Appendix A. Concrete Program Example

To make the state-machine representation concrete, we present a compiled program for the AndroidWorld `ContactsNewContactDraft` task (parameters: first name, last name, phone number, phone label).

```
{
  "metadata": {
    "task_description": "Go to the new contact screen and enter the following details",
    "application_context": "com.google.android.contacts",
    "parameters": ["first_name", "last_name", "phone_number", "phone_label"],
    "dedup_signature": "contacts_new_draft_4param"
  },
  "states": [
    {"id": "home", "verification": {"type": "expect_element", "xpath": "resource_id=...launcher", "
     timeout_ms": 3000}},
    {"id": "contacts_main", "verification": {"type": "expect_element", "xpath": "package=com.google.
     android.contacts", "timeout_ms": 5000}},
    {"id": "new_contact_form", "verification": {"type": "expect_element", "xpath": "resource_id=name_field
     ", "timeout_ms": 3000}},
    {"id": "form_filled", "verification": {"type": "expect_element", "xpath": "text=$first_name $last_name
     ", "timeout_ms": 2000}},
    {"id": "draft_saved", "verification": {"type": "terminal_state"}}
  ],
  "transitions": [
    {"from": "home", "to": "contacts_main",
     "action": {"type": "open_app", "app_name": "Contacts",
                "description": "open the Contacts app"}},
    {"from": "contacts_main", "to": "new_contact_form",
     "action": {"type": "action_click", "target": "content_desc=Create new contact",
                "description": "tap the Create new contact button"}},
    {"from": "new_contact_form", "to": "form_filled",
     "action": {"type": "input_text", "text": "$first_name $last_name", "index": 1,
                "description": "type the contact's first and last name (parameters)"}},
    /* ... three more transitions for phone number, label, save ... */
    {"from": "form_filled", "to": "draft_saved",
     "action": {"type": "action_click", "target": "content_desc=Save",
                "description": "tap Save to commit the new contact draft"}}
  ],
  "human_interventions": []
}
```

The verification predicate of each state is an XPath/accessibility-tree pattern that must hold on the live UI; replay halts and falls through to CUA if a predicate fails. Parameters ($first_name, $phone_number, etc.) are inferred at compile time from the live trace: any input text that matches a task-description placeholder is parameterized so the program generalizes across instances. Every transition carries a one-sentence `description` field that the selector reads when judging whether the program applies to a new task.

The state graph here is tree-like, but more complex tasks (delete-confirmation dialogs, navigation that branches on app version) compile into graphs with multiple outgoing transitions per state, where the state's verification predicate selects which branch is active.

Appendix B. Computer-Action Schema

Each transition's `action` field must be one of the action types in Table 10. Action types are translated to platform-specific backends at execution time: `action_click` on Android maps to a `JSONAction`-formatted touch with the resolved element's (x, y) coordinates; on OSWorld it maps to `pyautogui.click(x, y)`. The cross-platform schema enables the same compiled state-machine program to run on either backend, although in practice each task's compile is platform-specific because the verification predicates use platform-specific selector dialects (Android XPath vs. pyautogui screenshot regions).

Table 10: Computer action types in PreAct's program schema. Each transition uses exactly one type.

Action type	Description and required fields
`action_click`	Click on a UI element. `target`: selector.
`action_long_press`	Long-press on a UI element. `target`: selector.
`input_text`	Type text into a focused element. `text`: literal or $param. `index`: optional element id.
`action_keypress`	Send a key event. `key`: `Enter`, `Back`, `Tab`, etc.
`scroll`	Scroll a region. `direction`: `up/down/left/right`. Optional `index` for sub-element scroll.
`open_app`	Launch an application by name. `app_name`: string.
`wait`	Sleep for the platform's default delay.
`navigate_back`	Send the system Back gesture (Android) or browser-back equivalent.
`navigate_home`	Return to the home screen / desktop.
`inspect_text`	Read the value at a selector. `store_result_as`: variable name. Used by QA-style tasks.
`answer`	Emit a final textual answer. `text`: literal or computed.
`status`	Terminate the program. `goal_status`: `complete/infeasible`.

The `description` field on each transition (one short sentence) is consumed by the agentic Program Selector (§3) when judging whether a stored program applies to the current task. Selectors are written to be parameter-aware (e.g. "type the filename (parameter `filename`)" rather than "type text"), since the program's `parameters` list is what the LLM uses to bind the program's slots to the new task's instance values at retrieval time.

Appendix C. Per-Task Multi-Seed Data

Table 11 reports per-task warm-pass results across the three Gemini 3 Flash seeds of the cold→warm monotonicity experiment (§4.2, Table 2; seed totals 11/11/10 match that table exactly). The stable failure set (BrowserDraw, SystemBrightnessMax, SystemWifiTurnOn) fails across all three seeds; the remaining cross-seed variation falls on three sampling-sensitive tasks (BrowserMaze, CameraTakeVideo, ContactsAddContact), where the action sequence is sensitive to LLM sampling.

The complete per-task results across all 32 Android multi-seed runs and 8 OSWorld runs are also available in the project's `paper/findings_summary.md`.

Appendix D. Container Patch

The `android_server_patched.py` adds a `/state` endpoint to `huggingface/android_world:latest` that returns base64-encoded screenshots plus serialized accessibility-tree elements, sidestepping the upstream populated-AVD a11y-gRPC initialization wedge.

Table 11: Per-task warm-pass results across the three Gemini 3 Flash seeds on AndroidWorld official-15 (cold→warm monotonicity experiment, §4.2). Seed totals (11/11/10) match Table 2.

Task	seed=42	seed=100	seed=1337
AudioRecorderRecordAudio	PASS	PASS	PASS
AudioRecorderRecordAudioWithFileName	PASS†	PASS	PASS†
BrowserDraw	FAIL	FAIL	FAIL
BrowserMaze	PASS	PASS	FAIL
CameraTakePhoto	PASS	PASS	PASS
CameraTakeVideo	FAIL	FAIL	PASS
ClockStopWatchPausedVerify	PASS	PASS	PASS
ClockStopWatchRunning	PASS	PASS	PASS
ContactsAddContact	PASS†	PASS†	FAIL
ContactsNewContactDraft	PASS	PASS	PASS
FilesDeleteFile	PASS	PASS	PASS
MarkorCreateFolder	PASS†	PASS†	PASS†
MarkorCreateNote	PASS	PASS	PASS
SystemBrightnessMax	FAIL	FAIL	FAIL
SystemWifiTurnOn	FAIL	FAIL	FAIL
Total	11/15	11/15	10/15

† Live evaluator passes but the verify-before-store gate rejected the task's recompiled program on that seed (`verify_score=0`); the warm run still passed via RPA/hybrid/CUA.

Appendix E. Out-of-Distribution Generalization (Details)

To test whether the corpus transfers beyond seen tasks, we built a rag_db from the OSWorld test_tiny set (6 tasks across Chrome, LibreOffice Calc, LibreOffice Writer; 4 verify-gate-stored programs after compile filtering) and ran on `test_small_ood.json`: the 33 OSWorld test_small tasks that are *not* in test_tiny, spanning 9 domains (chrome×2, gimp×2, libreoffice_calc×1, libreoffice_impress×2, multi_apps×17, os×2, thunderbird×2, vlc×2, vs_code×3). The agentic Program Selector retrieves from the test_tiny corpus when the new task's description plausibly matches.

Result at n=3 reps (rag_db copy reset per rep): warm-OOD 21/30, 15/30, 14/30 = **mean** 16.7/30 = 55.6% ± 12.0. The cold-OOD baseline on the same 30 OOD tasks is a single (n=1) pass at approximately 20/30 = 66.7%, so warm-OOD is roughly 11 percentage points *below* cold-OOD (the cold side is unpaired and not replicated, so the magnitude is indicative).

The corpus does not transfer cleanly across task families. Inspecting the rep logs: the selector retrieves a near-miss program (a test_tiny Chrome history-clean program retrieved for a Thunderbird email task whose intent shares lexical surface with "clean") on ∼ 6/30 queries per rep on average; the source-domain XPath/state predicates then fail on the OOD page, consuming replay budget that the harness can no longer recover within the per-task wall budget. The remaining ∼ 24/30 OOD queries are handled correctly by the selector returning "no candidate"—these run as pure CUA and approximately match cold-OOD on their own—so the aggregate 11 pp regression is concentrated in the small fraction of mis-routed retrievals. *The corpus is not neutral on OOD; it is mildly harmful.* This sharpens the scope of the self-extending corpus claim: corpora built on small in-distribution task subsets should not be deployed cross-domain without retraining the selector or extending the corpus to cover the new domain.

Appendix F. Limitations (Extended)

(L1) Architectural baseline. We ran an ActionEngine-style flat-script baseline at n=5 Android seeds (Table 13) by gating per-state verification under a `PREACT_RUNTIME_MODE=flat_script` env var. Flat-script mean (warm) is 10.6 ± 1.14 across seeds 42/100/1337/7/2025, vs. state-machine 11.67 ± 0.58 on the matched 3 seeds; the matched-seed $\Delta = +0.67$ tasks in favor of state-machine, sign-test $p = 0.125$ on n=3 paired comparisons (suggestive but not significant). The architectural commitment matters but is a smaller effect than the verify-gate's 1.75–2.6-task diff-of-deltas, consistent with the paper's framing that the gate is the bigger empirical contribution. We did not run an untargeted-exploration baseline. The Gemini-vs-Claude compile-LLM concern previously flagged here is now closed at n=3 (§4.6).

(L2) Benchmark coverage. OSWorld test_tiny has only 6 tasks, AndroidWorld official-15 has 15, WebArena shopping_admin subset has 12. While the verify-gate effect replicates cross-platform, the benchmarks themselves are a small sample of computer-using tasks. We acknowledge platform-coverage bias as out-of-scope for this paper. The relative percentage-point magnitudes are influenced by the denominator: 43 pp on OSWorld's 6-task subset is mathematically larger than 17 pp on Android's 15-task subset and 15 pp on WebArena's 12-task subset, for absolute task counts that fall in a similar range (1.75–2.6 tasks).

(L3) Compile cost. The verify-before-store gate requires a full re-replay and re-evaluation of every compiled program. From timing measurements across our experiments (60 OSWorld tasks; 60 Android tasks under gate-ON), the verify-replay phase adds median 141 s on OSWorld and 76 s on Android per stored task, on top of the original CUA execution (median 65 s OSWorld; 47 s Android). This corresponds to **+217% wall overhead on OSWorld and +162% on Android per successful CUA-then-compile cycle** (verify-replay is a near-fixed cost roughly comparable to or larger than the original CUA execution on both platforms; it is proportionally largest on OSWorld, where both the absolute verify-replay time and the base CUA time are highest). The cost is amortized on warm runs where the corpus retrieves already-verified programs in seconds, but the compile-time overhead is real and may matter for high-throughput deployments.

(L4) Compile-fidelity rate. A separate audit of stored programs across platforms (Android: 58 programs; OSWorld: 16; WebArena: 7) found two recurring compile-fidelity risk patterns: missing `navigate_back` on nav-heavy edit/delete tasks (affecting roughly 28% of Android programs), and dynamic `inspect_text`/`inspect_screenshot` returns on pages whose state evolves between Run 1 and replay (100% of audited WebArena programs use `inspect_screenshot`, fully explaining the 48/48 smoking-gun rate observed under gate-OFF). The compiler itself is an LLM that produces lossy state-machines on a non-trivial fraction of inputs. The verify-gate filters this at storage time, but as the corpus scales, the gate's verify-replay cost (proportional to compile attempts, not just stored programs) may become a bottleneck.

(L5) LLM nondeterminism in selector. The agentic Program Selector is itself an LLM call. We measured selector exact-id-match accuracy of 22/30 = 73% on the rag_db's stored programs, prompted with each program's own task description verbatim. Inspecting the 8 misses, all were picks of *semantically equivalent* programs (same `task_description`, different `program_id` from accumulated reps under different dedup signatures). Functional accuracy (correct task family selected) is therefore close to 100%, with the 73% exact-id figure reflecting the corpus's accumulation of multiple matches per task.

To stress-test the selector beyond verbatim retrieval, we ran a paraphrase-robustness audit: for each of 15 stored Android programs, we generated 3 LLM-rewritten paraphrases (45 paraphrased queries total) and measured whether the selector still picked the original program. Result: 21/45 = 47% exact-id, 13/45 = 29% same-task-different-id (paraphrase picked an equivalent program), 11/45 = 24% no-pick (selector returned "no candidate"), and **0/45 = 0% wrong-task-family** (selector never picked an unrelated program). *Functional accuracy* (exact + equivalent) is 75.6% (34/45); the no-pick fraction is conservative behavior (the selector errs toward CUA fallback rather than retrieving an inapplicable program). At 75.6% functional retrieval and 0% mis-retrieval, the selector is conservative-by-design rather than over-eager.

Appendix G. Negative-Findings Tables

This appendix provides the per-seed tables for the negative findings summarized in §4.7.

Table 12: Code-level guardrails ablation, AndroidWorld official-15, n=5. Cold means are *identical* (10.2 = 10.2). Warm Δ = +0.4 is well within the standard deviations on each side.

Seed	Cold ON	Warm ON	Δ ON	Cold OFF	Warm OFF	Δ OFF
42	10	11	+1	10	12	+2
100	11	11	0	11	10	−1
1337	9	11	+2	10	10	0
2024	12	11	−1	10	10	0
7777	9	11	+2	10	11	+1
Mean	**10.2**	**11.0**	$+0.8 \pm 1.30$	**10.2**	**10.6**	$+0.4 \pm 1.14$

Table 13: Architectural ablation: state-machine vs. flat-script runtime, AndroidWorld official-15 warm SR. State-machine n=3 (matched seeds), flat-script n=5 (matched seeds + 7 + 2025).

Mode	seed=42	seed=100	seed=1337	seed=7	seed=2025	Mean ± std
state_machine (default)	12/15	12/15	11/15	—	—	**11.67 ± 0.58** (n=3)
flat_script (ActionEngine-style)	11/15	12/15	10/15	11/15	9/15	**10.6 ± 1.14** (n=5)
Δ (matched)	+1	0	+1	—	—	**+0.67 (matched n=3)**

Appendix H. Smoking-Gun Reproducibility (Per-Program)

The five smoking-gun cov=100%/score=0 cases (Table 5) are not equally reproducible across reps. Table 14 reports the per-program reproducibility across the 5 OSWorld warm-OFF reps.

Table 14: Smoking-gun reproducibility across 5 OSWorld warm-OFF reps. Each row: how many of 5 reps replayed the program at cov=100% and got score=0.

Program ID	Task	Reps with cov=100%/score=0
43188217	LibreOffice Calc formula	5/5 (fully deterministic)
c1f04f87	LibreOffice Calc chart	4/5 (rep2 hybrid-rescued)
8f0fdfa4	Chrome history clean	2/5 (rep3/4/5 replayed correctly)

The mechanism is partially deterministic. For the most reliable case (Calc formula 43188217), the lossy program reproducibly fails at cov=100% on every warm rep. For Calc chart c1f04f87, hybrid-mode rescue occasionally salvaged the run via CUA fallback after partial replay (cov=8%). For Chrome history 8f0fdfa4, the cold-stored program's mechanical action sequence happened to match the evaluator's required browser state on 3 of 5 reps—a partial-determinism that traces to environmental factors (Chrome's session state, browsing history initialization) varying across DesktopEnv resets even with identical task IDs.